



Rapport I5OM

Binôme : Thomas MORAUX – Ryan LAWSON

Enseignant : Jean-Luc BARIL

24 Novembre 2025



Sommaire

I. Transformée de Fourier discrète 2D.....	3
II. Transformée de Fourier rapide discrète 1D.....	5
III. Transformée de Fourier rapide discrète 2D.....	7
IV. Transformée de Fourier inverse rapide 1D.....	9
V. Transformée de Fourier inverse rapide 2D.....	11

I. Transformée de Fourier discrète 2D

1. La théorie :

$$F(g(x, y)) = \hat{g}(u, v) \\ = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} g(x, y) \exp(-2i\pi(\frac{ux}{M} + \frac{vy}{N}))$$

2. Le programme (Python) :

On présuppose que nous avons une matrice de taille M*N en entrée.

```
import cmath # Pour gérer les nombres complexes et exponentielles

def dft2d(input_matrix):
    M = len(input_matrix) # nombre de lignes
    N = len(input_matrix[0]) # nombre de colonnes
    output_matrix = [[0]*N for _ in range(M)] # matrice résultat

    for u in range(M):
        for v in range(N):
            sum_val = 0+0j
            for x in range(M):
                for y in range(N):
                    angle = -2j * cmath.pi * ((u*x)/M + (v*y)/N)
                    sum_val += input_matrix[x][y] * cmath.exp(angle)
            output_matrix[u][v] = sum_val
    return output_matrix
```

3. Explication du programme :

1. Le programme reçoit une matrice 2D *input_matrix* de dimensions M*N, représentant la valeur de l'image en chaque point (x,y). Les variables M et N représentent respectivement le nombre de lignes et de colonnes.

2. Le programme crée une matrice *output_matrix* $M \times N$ initialisée avec des zéros, qui contiendra le résultat de la transformation à chaque fréquence discrète (u,v) .
3. Pour chaque fréquence discrète (u,v) dans la plage $0 \leq u < M$, $0 \leq v < N$, on fait la sommation suivante :
 - On parcourt tous les points (x,y) de la matrice d'entrée.
 - Pour chaque point (x,y) , on calcule l'angle $-2\pi i(ux/M + vy/N)$
 - On calcule l'exponentielle complexe grâce à *cmath.exp(angle)*.
 - On multiplie la valeur originale *input_matrix[x][y]* par cette exponentielle complexe.
 - On accumule cette valeur dans *sum_val*.
 - Après avoir sommée toutes les contributions de tous les pixels (x,y) , on stocke *sum_val* dans *output_matrix[u][v]*.

Le résultat obtenu est une matrice $M \times N$ de nombre complexes représentant l'amplitude et la phase des composantes fréquentielles bidimensionnelles du signal d'origine.

Sa complexité est $O(M^2N^2)$ donc très lent pour de grandes matrices.

II. Transformée de Fourier rapide discrète 1D

1. La théorie :

A priori, la méthode de calcul naïve de la DFT revient à une multiplication matricielle, et a donc une complexité $\mathcal{O}(N^2)$. En pratique, les logiciels de calcul scientifique utilisent un algorithme beaucoup plus rapide (qui a joué un grand rôle dans le succès de l'utilisation de l'analyse de Fourier en traitement d'images). Cet algorithme consiste à séparer u en parties plus petites, et à se ramener à la DFT de ces signaux plus petits.

Considérons par exemple le cas où $N = 2K$ pour $K \in \mathbb{N}^*$. Notons pour tout $y \in [0 : K - 1]$,

$$\begin{cases} v(y) = u(2y) \\ w(y) = u(2y + 1) \end{cases}$$

Ainsi, pour tout $\xi \in \mathbb{Z}$,

$$\begin{aligned} \hat{u}(\xi) &= \sum_{x=0}^{N-1} u(x) e^{-2i\pi \frac{\xi x}{N}} = \sum_{y=0}^{K-1} u(2y) e^{-2i\pi \frac{\xi 2y}{N}} + \sum_{y=0}^{K-1} u(2y+1) e^{-2i\pi \frac{\xi (2y+1)}{N}} \\ &= \sum_{y=0}^{K-1} u(2y) e^{-2i\pi \frac{\xi y}{K}} + e^{-2i\pi \frac{\xi}{N}} \sum_{y=0}^{K-1} u(2y+1) e^{-2i\pi \frac{\xi y}{K}} = \hat{v}(\xi) + e^{-2i\pi \frac{\xi}{N}} \hat{w}(\xi). \end{aligned}$$

Par une décomposition en arbre binaire, on peut ramener le calcul de la transformée de Fourier discrète de N^2 à $N \cdot \log_2(N)$.

2. Le programme (Python) :

On présuppose que nous avons une matrice de taille N en entrée, avec N une puissance de 2 pour que l'algorithme fonctionne.

```
import cmath

def fft(x):
    N = len(x)
    if N == 1:
        return x
    else:
        even = fft(x[0::2])
        odd = fft(x[1::2])
        T = [cmath.exp(-2j * cmath.pi * k / N) * odd[k] for k in range(N // 2)]
        return [even[k] + T[k] for k in range(N // 2)] + \
               [even[k] - T[k] for k in range(N // 2)]
```

3. Explication du programme :

1. L'algorithme divise la liste d'entrée x représentant la valeur du signal en chaque point en 2 sous-listes :
 - Les éléments d'indice pair (liste *even*),
 - Les éléments d'indice impair (liste *odd*).
2. Ce découpage est récursif, la fonction *fft* s'appelle elle-même sur ces sous-listes jusqu'à atteindre la taille 1 (cas de base) :
 - Lorsque la taille du signal est 1, la DFFT est triviale : la transformée est le signal lui-même.
3. Pour chaque sous-liste (pair ou impair), on calcule la DFFT récursive. Ces DFFT sont les transformées partielles des moitiés du signal.
4. Après avoir obtenu les DFFT des parties paire et impaire, on calcule les coefficients complexes appelés "twiddle factors" qui correspondent à des rotations du plan complexe.

$$e^{-2i\pi \frac{k}{N}}$$

5. Chaque élément de la DFFT impaire est multiplié par son facteur correspondant.
6. On recompose la DFFT complète de taille N en combinant les résultats des deux sous-listes :

$$\text{Pour } k \in \left[0, \frac{N}{2} - 1\right] \quad \begin{cases} X(k) = Y(k) + w^k Z(k) \\ X(k + N/2) = Y(k) - w^k Z(k) \end{cases}$$

Le résultat obtenu est une matrice N de nombre complexes représentant l'amplitude et la phase des composantes fréquentielles unidimensionnelles du signal d'origine.

Cette version de l'algorithme nous permet de passer d'une complexité de $O(N^2)$ à $O(N \cdot \log(N))$ notamment via l'utilisation de la récursivité et de la technique « diviser pour régner ». Cela permet un gain de temps considérable.

III. Transformée de Fourier rapide discrète 2D

1. La théorie :

La transformée de Fourier discrète bidimensionnelle est la composée des deux transformées de Fourier unidimensionnelles en abscisse et en ordonnées.
La complexité est au plus de l'ordre de $O(M^2 N^2)$

2. Le programme (en Python) :

On présuppose que nous avons une matrice de taille $N \times N$ en entrée, avec N une puissance de 2 pour que l'algorithme fonctionne.

```
def fft2d(matrix):  
    N = len(matrix)  
    # FFT sur les lignes  
    temp = [fft(row) for row in matrix]  
    # Transposer le résultat  
    temp = [list(row) for row in zip(*temp)]  
    # FFT sur les colonnes  
    temp = [fft(row) for row in temp]  
    # Transposer pour revenir à l'orientation originale  
    result = [list(row) for row in zip(*temp)]  
    return result
```

3. Explication du programme :

1. On récupère d'abord la taille de la matrice *matrix*, représentant la valeur de l'image ou du signal spatial en chaque point (x,y), qu'on stocke dans N .

2. Puis, une DFFT 1D, via la réutilisation de la fonction *fft(x)* présentée dans le II, est appliquée sur chaque ligne de la matrice. Cela transforme chaque ligne du domaine spatial au domaine fréquentiel sur une seule dimension.
3. Pour pouvoir appliquer la DFFT sur les colonnes, la matrice transformée est transposée. Cette transposition échange les lignes et colonnes, ce qui permet ensuite de traiter les anciennes colonnes comme des lignes car en Python il est plus simple de transformer les colonnes en lignes pour réutiliser la même fonction, ce qui est fait grâce à la transposition avec *zip(*row)* regroupant les éléments selon leur index (toutes les premières valeurs ensemble, etc), ce qui crée des groupes représentant les colonnes de la matrice d'origine. *list(row)* convertit le résultat de zip en une liste
4. Ensuite, la DFFT 1D, toujours via la même fonction *fft(x)*, est appliquée sur ces nouvelles « lignes » (anciennes colonnes), ce qui équivaut à réaliser la DFFT sur les colonnes originales.
5. Enfin, on retranspose pour remettre la matrice dans sa forme originale.

Le résultat obtenu est une matrice $N \times N$ de nombre complexes représentant l'amplitude et la phase des composantes fréquentielles bidimensionnelles du signal d'origine.

Cette version de l'algorithme nous permet de passer d'une complexité de $O(N^4)$ à $O(2 \cdot N^2 \cdot \log(N))$ notamment via l'utilisation de la récursivité et de la technique « diviser pour régner » mais aussi du raccourcis mathématique consistant à appliquer la DFFT 1D sur les lignes puis sur les colonnes de la matrice pour obtenir la DFFT 2D. Cela permet un gain de temps considérable.

IV. Transformée de Fourier inverse rapide 1D

1. La théorie :

Inverse :

$$F'(\hat{g}(u)) = g(x) = \sum_{u=0}^{N-1} \hat{g}(u) \exp\left(\frac{2i\pi ux}{N}\right) \text{ avec } x = 0..N-1$$

Par une décomposition en arbre binaire, on peut ramener le calcul de la transformée de Fourier discrète de N^2 à $N \cdot \log_2(N)$.

2. Le programme (en Python) :

On présuppose que nous avons une matrice de taille N en entrée, avec N une puissance de 2 pour que l'algorithme fonctionne.

```
import cmath

def ifft(x):
    N = len(x)
    if N <= 1:
        return x
    even = ifft(x[0::2])
    odd = ifft(x[1::2])
    T = [cmath.exp(2j * cmath.pi * k / N) * odd[k] for k in range(N // 2)]
    return [even[k] + T[k] for k in range(N // 2)] + \
           [even[k] - T[k] for k in range(N // 2)]
```

3. Explication du programme :

Ce programme réutilise le même algorithme, les mêmes techniques que celui de base de la DFFT 1D. La différence réside dans le changement du signe au niveau de l'exponentiel dans le "twiddle factors" passant d'un - à un +.

Le résultat obtenu est une matrice N de nombre représentant le signal originel.

Cette version de l'algorithme nous permet de passer d'une complexité de $O(N^2)$ à $O(N \cdot \log(N))$ notamment via l'utilisation de la récursivité et de la technique « diviser pour régner ». Cela permet un gain de temps considérable par rapport à la IDFT.

V. Transformée de Fourier inverse rapide 2D

1. La théorie :

La transformée de Fourier discrète bidimensionnelle est la composée des deux transformées de Fourier unidimensionnelles en abscisse et en ordonnées.
La complexité est au plus de l'ordre de $O(M^2 N^2)$

2. Le programme (en Python) :

On présuppose que nous avons une matrice de taille $N \times N$ en entrée, avec N une puissance de 2 pour que l'algorithme fonctionne.

```
def ifft2d(matrix):  
    N = len(matrix)  
    # IFFT sur les lignes  
    temp = [ifft(row) for row in matrix]  
    # Transposer le résultat pour appliquer la IFFT sur les colonnes  
    temp = [list(row) for row in zip(*temp)]  
    # IFFT sur les colonnes  
    temp = [ifft(row) for row in temp]  
    # Retransposer pour revenir à la forme originale  
    result = [list(row) for row in zip(*temp)]  
    return result
```

3. Explication du programme :

L'algorithme utilisé pour ce programme est exactement le même que celui décrit dans III.

Le résultat obtenu est une matrice $N \times N$ de nombre représentant le signal bidimensionnel ou l'image originel.

Cette version de l'algorithme nous permet de passer d'une complexité de $O(N^4)$ à $O(2*N^2*\log(N))$ notamment via l'utilisation de la récursivité et de la technique « diviser pour régner » mais aussi du raccourcis mathématique consistant à appliquer la IDFFT 1D sur les lignes puis sur les colonnes de la matrice pour obtenir la IDFFT 2D. Cela permet un gain de temps considérable.